

TrashUQ: Deep Dive tècnic (en català)

1. Què és TrashUQ i quin problema resol

TrashUQ és una plataforma **Edge AI + Federated Learning** per classificar residus en temps real i monitoritzar tot el sistema de punta a punta.

Objectiu principal: - Classificar material (**cardboard, glass, paper, plastic**) a dispositius de vora. - Fer-ho amb latència baixa i consum controlat. - Millorar el model amb dades locals sense centralitzar imatges sensibles. - Tenir observabilitat completa (estat, mètriques, esdeveniments, rondes FL).

Per això el projecte combina: - **edge/**: inferència local i flux de captura - **mqtt/**: telemetria en temps real - **backend/**: ingestió, persistència i coordinació FL - **frontend/**: dashboard en viu - **experiments/part_b/**: validació i escalabilitat FL

2. Arquitectura global: per què està separada així

Hi ha **dos plans de comunicació** diferents perquè resolen necessitats diferents.

1. Pla de telemetria (MQTT)

- Què fa: envia estats, mètriques i esdeveniments en streaming.
- Per què MQTT: protocol lleuger, ideal per edge, topologia pub/sub i integració fàcil amb dashboard en viu.
- Com: els clients publiquen a `arduino/<device_id>/...`, backend subscriu `arduino/+/#` i frontend també escolta via WebSocket MQTT.

2. Pla de Federated Learning (gRPC)

- Què fa: coordina `Join`, obtenció de model global i enviament d'updates.
- Per què gRPC: contracte tipat, serialització eficient, vectors numèrics ben definits per rondes iteratives.
- Com: `edge/bin_mpu/fl_client.py` parla amb `backend/app/grpc_server.py` sobre `fl.proto`.

Aquesta separació evita barrejar tràfic “freqüent i petit” (telemetria) amb tràfic “estructurat i crític de model” (FL).

3. Com funciona cada part del repositori

- `compose.yaml`
 - Orquestra `db, mqtt, backend, frontend`.
 - Exposar REST (4000), gRPC (50051), MQTT (1883), MQTT WS (9001), frontend (3000).
- `backend/`

- `app/main.py`: arrencada FastAPI + gRPC + MQTT ingest.
- `app/mqtt_runtime.py`: subscriber MQTT del backend.
- `app/service.py`: parseig/normalització de missatges i construcció del bootstrap del dashboard.
- `app/db.py`: crea esquema PostgreSQL.
- `app/fl_coordinator.py`: lògica FL (estat global, agregació, control de concurrència).
- `app/fl.proto`: contracte gRPC.
- `edge/`
 - `bin_mpu/pipeline.py`: màquina d'estats del contenidor intel·ligent.
 - `bin_mpu/classifier.py`: inferència TFLite + votació per burst + calibració.
 - `bin_mpu/finetuner.py`: entrenament local del capçal petit en NumPy.
 - `bin_mpu/fl_client.py`: cicle de ronda FL via gRPC.
- `frontend/`
 - `lib/mqtt.ts`: connexió MQTT del navegador.
 - `app/page.tsx`: normalitza dades i pinta mètriques, clients i rondes.
- `experiments/part_b/`
 - `run_part_b.py`: simulacions controlades per avaluar comportament FL amb diferents nombres de clients.

4. Pipeline edge: com hem dissenyat la lògica operativa

La màquina d'estats (`edge/bin_mpu/pipeline.py`) és: - IDLE - CAPTURING - WAITING_LABEL

Flux: 1. Trigger PIR -> CAPTURING. 2. Captura burst (5 frames). 3. Inferència per frame + votació majoritària. 4. Si confiança $\geq$ llindar: - guarda mostra com `model` - publica classificació MQTT - obre tapa si coincideix amb la classe del contenidor 5. Si confiança $<$ llindar: - guarda mostra `model_uncertain` - passa a WAITING_LABEL - espera etiqueta humana 6. Quan arriba etiqueta humana: - desa `label_src='user'` - notifica FL per possible ronda local

Per què així: - Automatitza casos clars. - Converteix incertesa en dades útils. - El feedback humà tanca el loop d'aprenentatge real al dispositiu.

5. Xarxes neuronals: decisions i mecànica interna

5.1 Model base de classificació

Entrenat a `edge/model/train.py` amb transfer learning: - Backbone: MobileNetV2 preentrenat a ImageNet (`include_top=False`). - Capçal: -

GlobalAveragePooling2D - Dropout(0.3) - Dense(128, relu) - Dropout(0.2)
- Dense(4, softmax)

Entrenament en 2 fases: 1. **Head training** amb backbone congelat. 2. **Fine-tuning** de capes superiors amb LR més baixa.

Per què aquesta estratègia: - Necessitàvem un model robust sense entrenar des de zero. - Transfer learning redueix dades necessàries i temps. - MobileNetV2 és eficient per edge i exportable a TFLite.

5.2 Inferència al dispositiu

edge/bin_mpu/classifier.py: - Carrega model TFLite. - Preprocessa imatge (RGB, resize, normalització [-1,1] en float32). - Obté probabilitats base p . - Aplica (si existeix) capçal de calibració per obtenir q .

A més, fa votació sobre burst de frames: - Menys sensibilitat a soroll puntual. - Més estabilitat de predicció en escenari real.

5.3 Per què FL no entrena tot el model

Decisió clau a edge/bin_mpu/finetuner.py: - **No** reentrenem MobileNetV2 complet a edge. - Només entrenem un capçal petit: - $q = \text{softmax}(W \cdot p + b)$

Amb 4 classes: - W és 4×4 - b és 4 - Total: **20 paràmetres**

Per què: - Cost computacional i energètic molt inferior. - Entrenament viable en hardware limitat. - Payload FL molt petit -> menys cost de comunicació.

Com s'entrena localment: - Recupera mostres etiquetades per usuari de SQLite (label_src='user'). - Passa cada imatge pel model congelat per obtenir p . - Entrena W, b amb SGD + cross-entropy en NumPy. - Retorna local_weights, num_samples, local_loss, local_accuracy.

Detall important de qualitat: - Si el model global rebut és vector tot zero, **no** s'aplica. - Motiu: un W zero faria sortides gairebé uniformes i degradaria inferència.

6. Federated Learning: com ho hem implementat i per què

El coordinador (backend/app/fl_coordinator.py) manté: - ronda actual - versió de model - pesos globals - clients online - updates pendents de la ronda

6.1 Algorisme d'agregació

Quan hi ha prou clients (min_clients_per_round), aplica una mitjana ponderada per mostres:

$w_{\text{global_nou}} = \text{sum}((n_i / N_{\text{total}}) * w_i)$

Per què ponderar per `num_samples`: - Un client amb més dades aporta més evidència estadística. - És el patró FedAvg clàssic.

Després: - incrementa `model_version` - incrementa `round` - neteja updates pendents

6.2 Control de concurrència i stale updates

Problema real: - múltiples clients poden enviar quasi al mateix temps.

Solució aplicada: - lock global amb `threading.Lock` - si `round` rebuda $\neq$ ronda actual $\rightarrow$ update rebutjada com stale

Per què és crític: - Evita barrejar updates de rondes diferents. - Garanteix coherència temporal del model global.

6.3 Coherència de dimensions

El finetuner edge envia $n^2 + n$ pesos (amb 4 classes = 20). Per tant, backend ha d'usar `FL_MODEL_SIZE=20` en desplegament coherent amb aquest capçal.

7. gRPC + FL: explicació precisa del funcionament

Contracte definit a `backend/app/fl.proto`.

RPCs: 1. `Join(client_id)` - registra client i retorna `round`, `model_version`, `global_weights`.

2. `GetGlobalModel(client_id)`

- retorna l'últim estat global abans d'entrenar localment.

3. `SubmitUpdate(client_id, round, num_samples, local_weights, local_loss, local_accuracy)`

- envia update local.
- resposta indica si s'ha agregat (`round_aggregated`) i l'estat actual.

Seqüència d'una ronda real

A `edge/bin_mpu/fl_client.py`: 1. `start()` crea canal gRPC i fa `Join`. 2. Cada etiqueta humana incrementa comptador local. 3. En arribar al llindar (`fl_trigger_user_samples`), inicia ronda en thread. 4. Fa `GetGlobalModel` per sincronitzar-se. 5. Executa entrenament local (`finetuner.run_round()`). 6. Envia `SubmitUpdate`. 7. Backend respon: - "acceptat, esperant més clients" o - "ronda agregada". 8. Client actualitza ronda/versió locals.

Per què aquesta seqüència: - sincronitza abans d'entrenar - evita entrenar sobre models obsolets - manté convergència i traçabilitat de versions

Per què gRPC aquí és bona decisió

- Contracte fort i tipat (evita ambigüitats JSON).
- Stubs generats compartits entre edge i backend.
- Missatges compactes per vectors numèrics repetits.
- Gestió explícita d'errors (`RpcError`) i timeouts.

Nota de seguretat: - Implementació actual usa `insecure_channel` (sense TLS), adequada per LAN controlada/prototip. - En entorns no confiables cal TLS i autenticació de clients.

8. Persistència i dashboard: com arribem a observabilitat completa

`backend/app/service.py` classifica missatges per topic i desa: - `mqtt_messages` (raw i JSON) - `device_status_latest` - `device_status_history` - `coordinator_metrics`

Després `GET /api/dashboard/bootstrap` construeix una vista agregada: - estat de dispositius - històric de mètriques - esdeveniments/logs/classificacions - snapshot FL (ronda, versió, pendents, etc.)

Per què aquest disseny: - El frontend no ha d'interpretar dades crues heterogènies. - El backend centralitza normalització i consistència semàntica.

9. Experiments FL (Part B): què valida i com

`experiments/part_b/run_part_b.py` executa simulacions controlades amb diferents nombres de clients i llavors.

Per què tenim aquest mòdul a part: - El runtime edge és validació operativa. - El mòdul Part B és validació científica/reproductible d'escalabilitat i convergència.

Genera: - CSVs de mètriques - figures - registres d'execució - informe markdown a `artifacts/part_b/latest/`

10. Decisió arquitectònica clau (resum del “per què”)

1. MQTT per telemetria

- baixa sobrecàrrega, pub/sub natural, temps real.

2. gRPC per FL

- contracte tipat, eficiència i robustesa en intercanvi de pesos.

3. Model base congelat + capçal petit entrenable

- FL viable a edge limitat, amb cost de càlcul i comunicació baix.

4. Human-in-the-loop només en incertesa

- minimitza fricció d'usuari i maximitza qualitat de noves dades.

5. FedAvg ponderat per mostres

- integra contribucions heterogènies de manera estadísticament raonable.

6. Control stale/concurrencia al coordinador

- preserva consistència de ronda i estabilitat del model global.
-

11. Conclusió tècnica

TrashUQ és una arquitectura coherent per portar IA de classificació a edge real, amb feedback humà, aprenentatge federat i observabilitat completa. El valor tècnic no és només el model, sinó la integració robusta entre inferència, dades, coordinació FL i monitorització operativa.

12. Q&A de defensa (preguntes típiques del professor)

1) Per què heu separat MQTT i gRPC?

Perquè resolten problemes diferents: MQTT és ideal per telemetria en temps real (pub/sub, lleuger), i gRPC és millor per intercanvi tipat i eficient de pesos i estat FL.

2) Per què no feu FL sobre tot el model MobileNetV2?

Per limitacions de càlcul i energia a edge. Entrenem només el capçal de calibració ($\mathbf{W}, \mathbf{b}$) per fer el procés viable i barat.

3) Quants paràmetres intercanvieu en FL?

Amb 4 classes, $n^2 + n = 20$ floats (16 de $\mathbf{W}$ + 4 de $\mathbf{b}$).

4) Quin algorisme federat useu?

FedAvg ponderat per `num_samples` de cada client.

5) Com controleu condicions de carrera al coordinador?

Amb `threading.Lock` i validació estricta de ronda; updates stale es rebutgen.

6) Què passa si dos clients envien ahora?

Es serialitza dins el lock; només s'agrega quan s'arriba al mínim de clients per ronda.

7) Què passa si un client envia una ronda antiga?

El backend retorna error de stale round i no altera el model global.

8) On persistiu les dades?

A PostgreSQL: missatges MQTT crus, estats de dispositiu (latest + history) i mètriques de coordinació.

9) El model global FL persisteix a DB?

No en aquesta versió: l'estat del coordinador és en memòria (es perd en reinici).

10) Com entra el feedback humà al sistema?

Quan la confiança és baixa, es demana etiqueta; aquesta mostra local etiquetada alimenta el fine-tuning i pot activar ronda FL.

11) Per què burst de 5 frames?

Per reduir soroll de prediccions puntuals: votació majoritària més robusta que una sola captura.

12) Quina és la funció del dashboard?

Observabilitat operacional i FL: estat dispositius, mètriques, events, classificacions, historial i estat federat.

13) Quina diferència hi ha entre validació desplegada i Part B?

Desplegada: prova end-to-end real. Part B: simulació controlada per escalar clients i estudiar convergència/cost.

14) Com gestioneu dades no vàlides?

El backend parseja i normalitza; si payload no és JSON vàlid o no té camps esperats, es conserva text cru i s'eviten trencaments.

15) La comunicació gRPC és segura?

Actualment no: `insecure_channel` (prototip/LAN). En producció cal TLS + autenticació.

16) Per què MobileNetV2?

Balanç bo entre precisió i eficiència per edge i ecosistema robust per exportar a TFLite.

17) Com eviteu degradar model local amb pesos globals dolents?

No s'aplica vector global tot-zero (cas inicial del coordinador) perquè destruiria la calibració útil local.

18) Què mesuren `local_loss` i `local_accuracy`?

Qualitat del capçal local després d'entrenament sobre mostres etiquetades per usuari del client.

19) Què passa si no hi ha prou mostres locals?

No es llança ronda (`run_round()` retorna `None` si hi ha menys de 2 mostres útils).

20) Quin és el punt fort principal del projecte?

Integració completa i coherent: edge real, telemetria, persistència, FL coordinat, dashboard en viu i evidència experimental.

13. Limitacions, riscos i com defensar-los

L1) Mida de vector FL

- **Risc:** desalineació entre `FL_MODEL_SIZE` backend i mida real del capçal edge.
- **Impacte:** updates rebutjades per longitud incorrecta.
- **Defensa:** fixar `FL_MODEL_SIZE=20` per 4 classes; si canvien classes, recalcular n^2+n .

L2) Estat FL en memòria

- **Risc:** reinici backend resetja ronda/versió/pesos globals.
- **Impacte:** pèrdua de continuïtat d'entrenament federat.
- **Defensa:** en aquesta fase és acceptable per prototip; següent pas és persistir snapshot FL a DB o object storage.

L3) gRPC sense TLS

- **Risc:** exposició en xarxes no confiables.
- **Impacte:** confidencialitat/integritat de missatges FL.

- **Defensa:** entorn de validació LAN controlat; roadmap inclou TLS mutu i autenticació de clients.

L4) Dependència de feedback humà

- **Risc:** si no hi ha etiquetes d'usuari, FL local avança lent.
- **Impacte:** menys adaptació personalitzada per client.
- **Defensa:** disseny intencional per qualitat de labels; opcionalment es pot afegir trigger temporal + pseudo-labeling controlat.

L5) Coordinador simple (single process)

- **Risc:** no hi ha alta disponibilitat ni distribució.
- **Impacte:** límit d'escalat en producció gran.
- **Defensa:** abast de TFG/prototip; arquitectura separada ja permet evolucionar a coordinador persistent/distribuït.

L6) Data drift i heterogeneïtat

- **Risc:** dades diferents per client poden desestabilitzar convergència.
- **Impacte:** variabilitat de rendiment entre bins.
- **Defensa:** precisament FL i ponderació per mostres minimitzen aquest efecte; Part B analitza comportament en escenaris no-IID.

14. Traça demo real (de sensor a ronda FL)

Escenari

- Dispositiu edge en mode actiu.
- Backend + MQTT + DB + frontend aixecats amb `docker compose`.
- FL activat al client edge.

Seqüència pas a pas

1. **PIR detecta objecte**
 - pipeline passa IDLE -> CAPTURING.
2. **Captura burst i infereix**
 - 5 frames, classificació per frame, votació majoritària.
3. **Decisió per confiança**
 - Si alta: classifica i (si toca) obre tapa.
 - Si baixa: desa mostra i entra a `WAITING_LABEL`.
4. **Usuari etiqueta mostra incerta**

- Mostra passa a `label_src='user'` a SQLite local.
 - Es publica event MQTT `label_received`.
5. **Trigger de FL local**
- Quan s'arriba al llindar de mostres noves:
 - FL client fa `GetGlobalModel`.
 - FineTuner entrena `W, b` en local.
6. **Submit gRPC al backend**
- Envia `SubmitUpdate(round, num_samples, local_weights, local_loss, local_accuracy)`.
7. **Coordinador avalua**
- Si falten clients: update acceptada pendent.
 - Si mínim assolit: FedAvg, increment de `model_version` i `round`.
8. **Observabilitat**
- Backend desa telemetria i mètriques.
 - Frontend actualitza charts/estat gairebé en viu per MQTT WS + bootstrap REST.

Missatge clau de defensa

Aquest flux demostra un cicle complet tancat: **captura -> decisió -> feedback humà -> aprenentatge local -> agregació federada -> monitorització**.

15. Checklist ràpid abans de la defensa

- FL_MODEL_SIZE coherent amb capçal (4 classes -> 20).
- Pots explicar diferència MQTT vs gRPC en 20 segons.
- Pots dibuixar de memòria el flux `Join -> GetGlobalModel -> SubmitUpdate`.
- Tens clara la limitació “estat FL en memòria” i la millora proposada.
- Saps justificar per què FL només sobre capçal petit i no model complet.
- Tens un exemple concret de com una etiqueta humana acaba impactant el model.